Pensamento Computacional
INF05008

Aula 13 – Dados Estruturados

Prof. Lucas Nunes Alegre

Universidade Federal do Rio Grande do Sul

Instituto de Informática

Departamento de Informática Teórica

Slides baseados no material da Prof. Leila Ribeiro.

Tipos Básicos

Até agora, vimos alguns tipos básicos:

- Number
  - 2, 45, 2.3, -1.0651
- Boolean
  - true, false
- String
  - “A”, “casa”, “10”
- Image





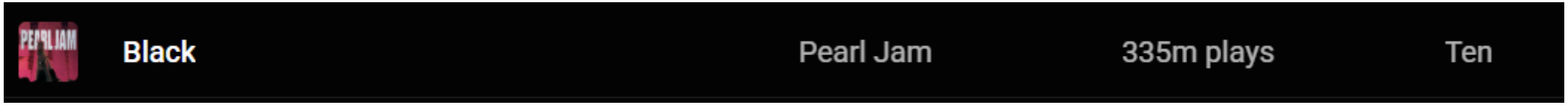
Como construir novos tipos?

Definir estruturas (registros)
é uma maneira!

Dados Estruturados

Em geral, queremos representar dados que possuem diversos atributos (partes).

- Uma música tem diversos atributos:
  - Um nome, um artista, um gênero, uma duração, ...



- Um email tem diversos atributos:
  - Um destinatário, assunto, mensagem, anexo, ...



- O mesmo vale para cadastros de pessoas, objetos, etc.

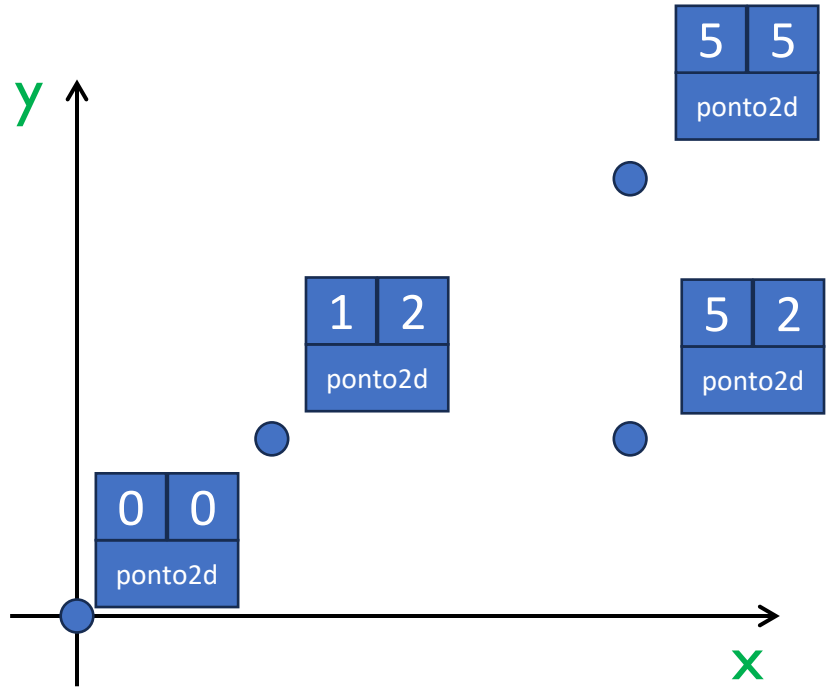
Estruturas

- Em linguagens de programação, usamos estruturas para representar dados não-atômicos, i.e., dados compostos de mais de um valor:
  - Um ponto no plano é composto de 2 coordenadas (x e y)
  - Um cadastro de aluno contém várias informações
  - Um objeto tem muitos atributos
  - ...

- Vamos agora fazer um programa para fazer operações com pontos 2D.

Dados Estruturados – Ponto 2D

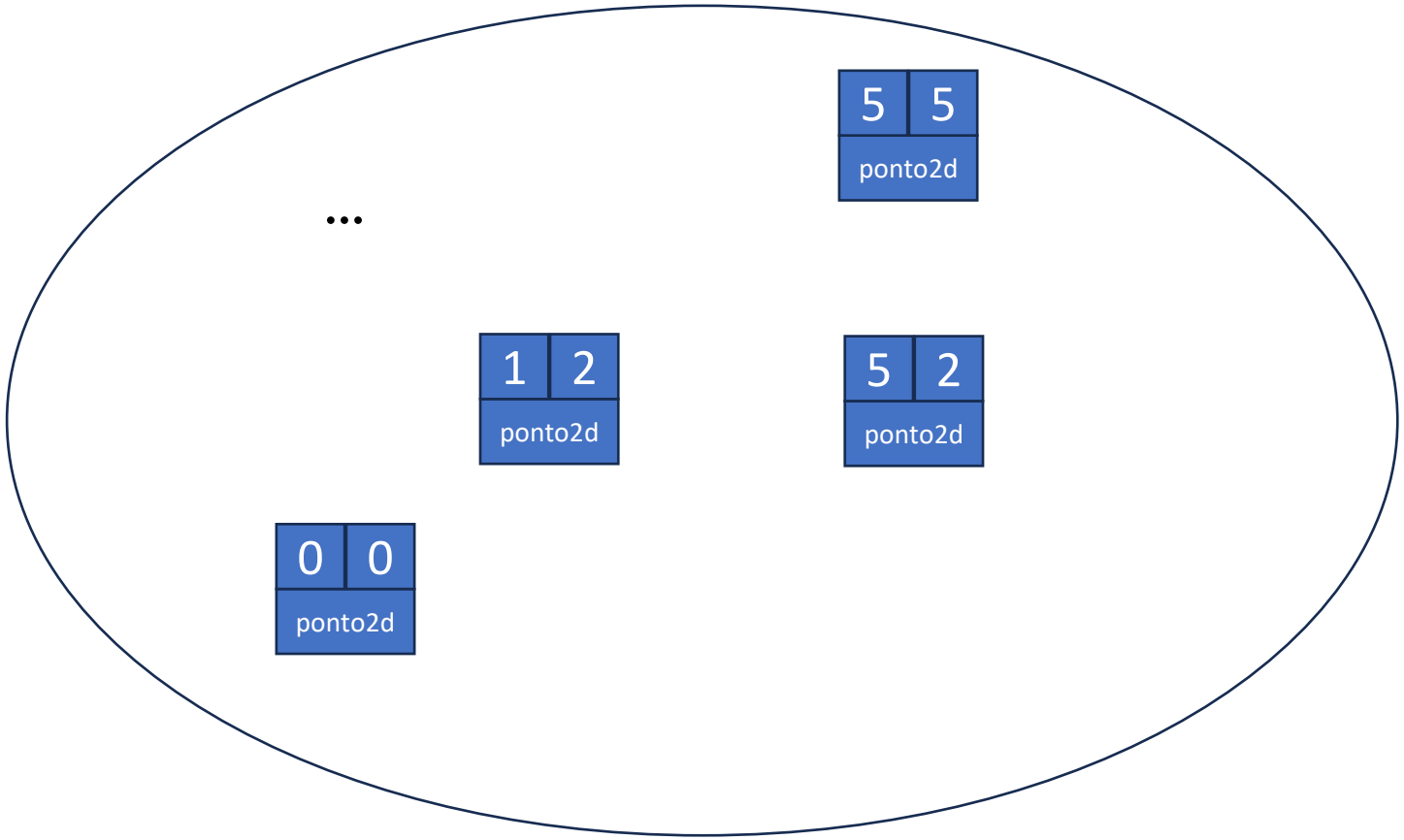
- Vamos fazer um programa para operar sobre pontos em um plano 2D.
  - Que informações precisamos guardar?



Um ponto2d tem
- coord-x
- coord-y

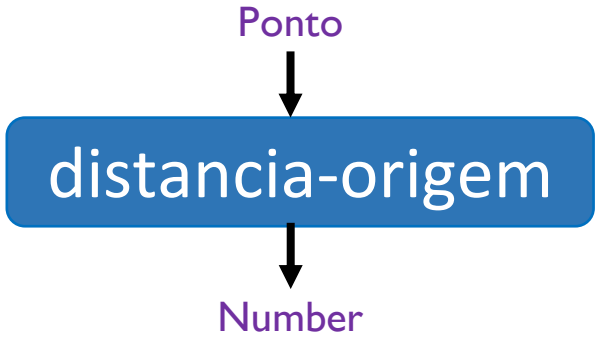
Dados Estruturados – Ponto 2D

- Ao definir um novo dado estruturado, estamos definindo um Conjunto!



distancia-origem

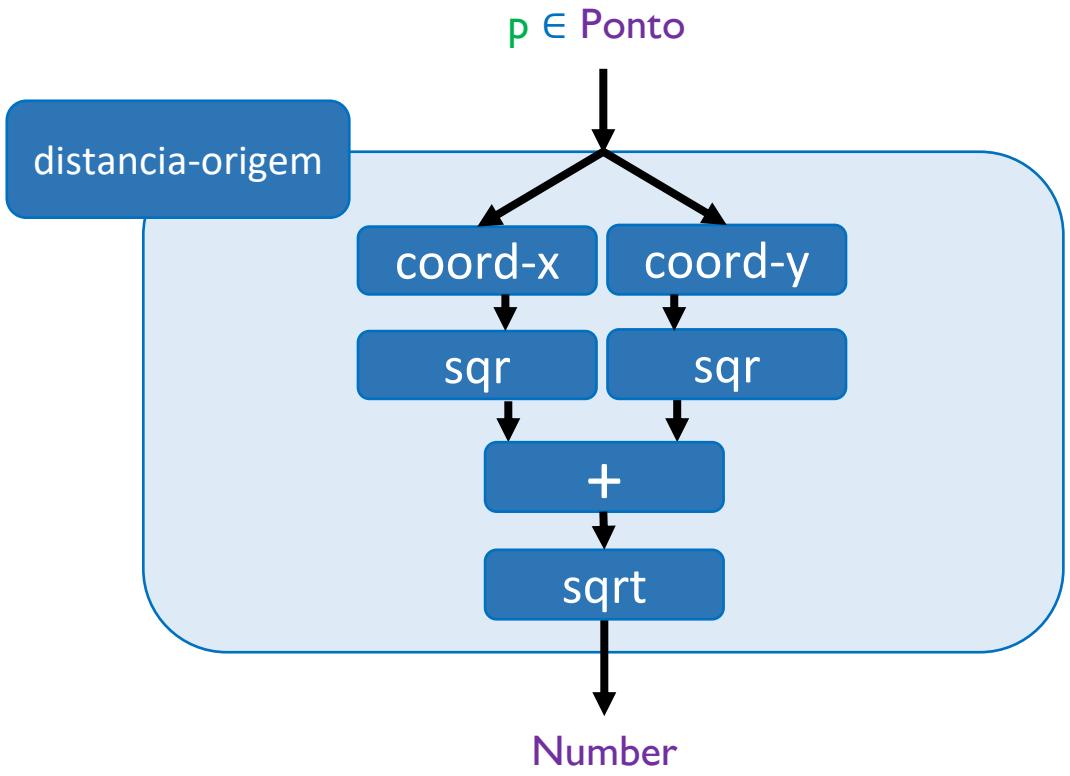
- Dado um ponto no plano, retorna a distância deste ponto à origem (ponto | 0 | 0 |).
- Exemplos:
  - distancia-origem(| 2 | 0 |) is 2
  - distancia-origem(| 3 | 2 |) is 3.6



$$\text{distancia-origem}(\text{ponto}) = \sqrt{(\text{coordenada x do ponto})^2 + (\text{coordenada y do ponto})^2}$$

distancia-origem

$$\text{distancia-origem}(\text{ponto}) = \sqrt{(\text{coordenada x do ponto})^2 + (\text{coordenada y do ponto})^2}$$



Para construir esse programa, precisamos definir o **Conjunto/Tipo Ponto**!

Definição do Conjunto **Ponto**

Para definir este conjunto/tipo, precisamos:

1. Definir o nome do tipo: **Ponto**.
2. Definir os atributos de cada elemento do conjunto **Ponto**

   e dar um nome para cada atributo.
3. Definir a forma sintática dos elementos do conjunto **Ponto**.
4. Definir o tipo de cada atributo de um elemento de **Ponto**.
5. Definir a semântica dos atributos de um **Ponto**

   (primeiro atributo representa a coordenada x e segundo a coordenada y).

Definição do Conjunto **Ponto**

```
data Ponto:
    # Um elemento do conjunto Ponto tem o formato:
    ponto2d(
        coord-x :: Number,  # Coordenada x do ponto
        coord-y :: Number   # Coordenada y do ponto
    )
end
```

Definição do Conjunto **Ponto**

1. Nome do Tipo

```
data Ponto:
    # Um elemento do conjunto Ponto tem o formato:
    ponto2d(
        coord-x :: Number,  # Coordenada x do ponto
        coord-y :: Number   # Coordenada y do ponto
    )
end
```

Definição do Conjunto **Ponto**

```
data Ponto:
    # Um elemento do conjunto Ponto tem o formato:
    ponto2d(
        coord-x :: Number,  # Coordenada x do ponto
        coord-y :: Number   # Coordenada y do ponto
    )
end
```

2. Nome dos Atributos

Definição do Conjunto **Ponto**

```
data Ponto:
    # Um elemento do conjunto Ponto tem o formato:
    ponto2d(
        coord-x :: Number,  # Coordenada x do ponto
        coord-y :: Number   # Coordenada y do ponto
    )
end
```

3. Forma sintática dos elementos do conjunto Ponto

Definição do Conjunto **Ponto**

```
data Ponto:
    # Um elemento do conjunto Ponto tem o formato:
    ponto2d(
        coord-x :: Number,  # Coordenada x do ponto
        coord-y :: Number   # Coordenada y do ponto
    )
end
```

4. Tipo dos Atributos

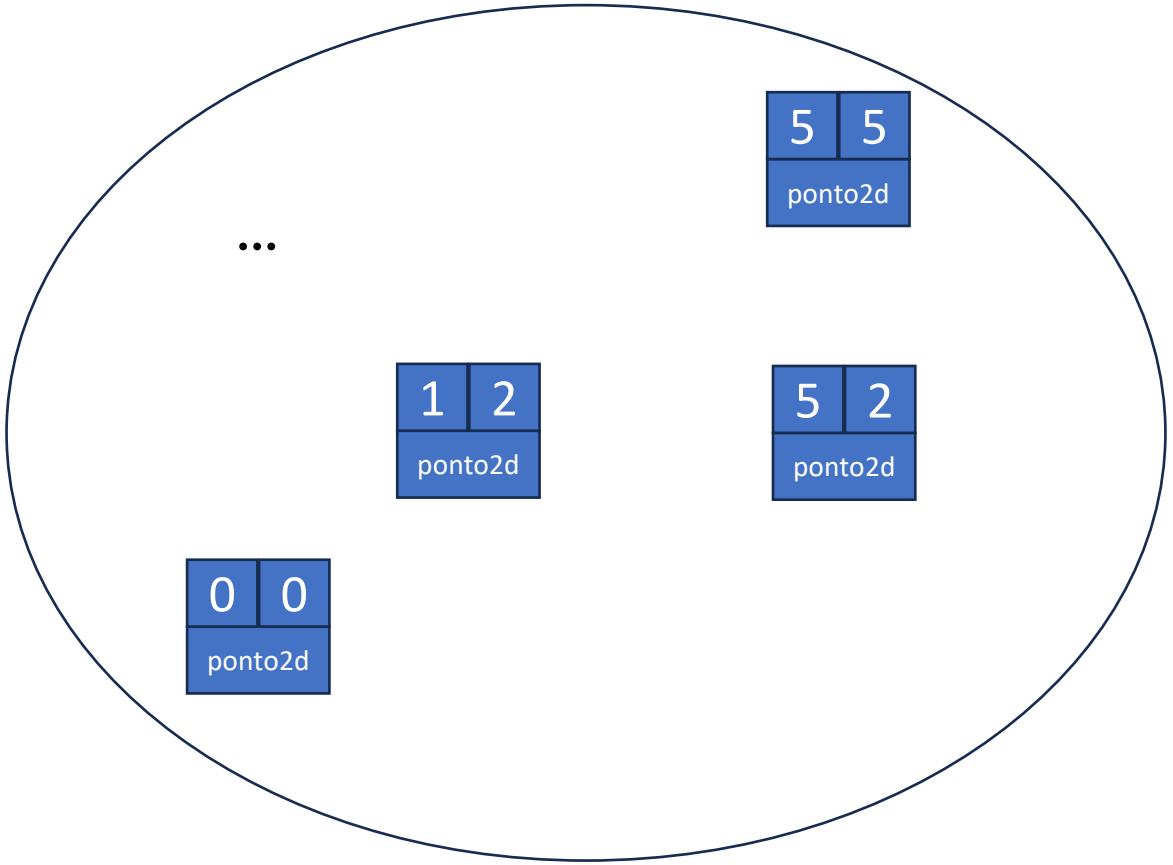
Definição do Conjunto **Ponto**

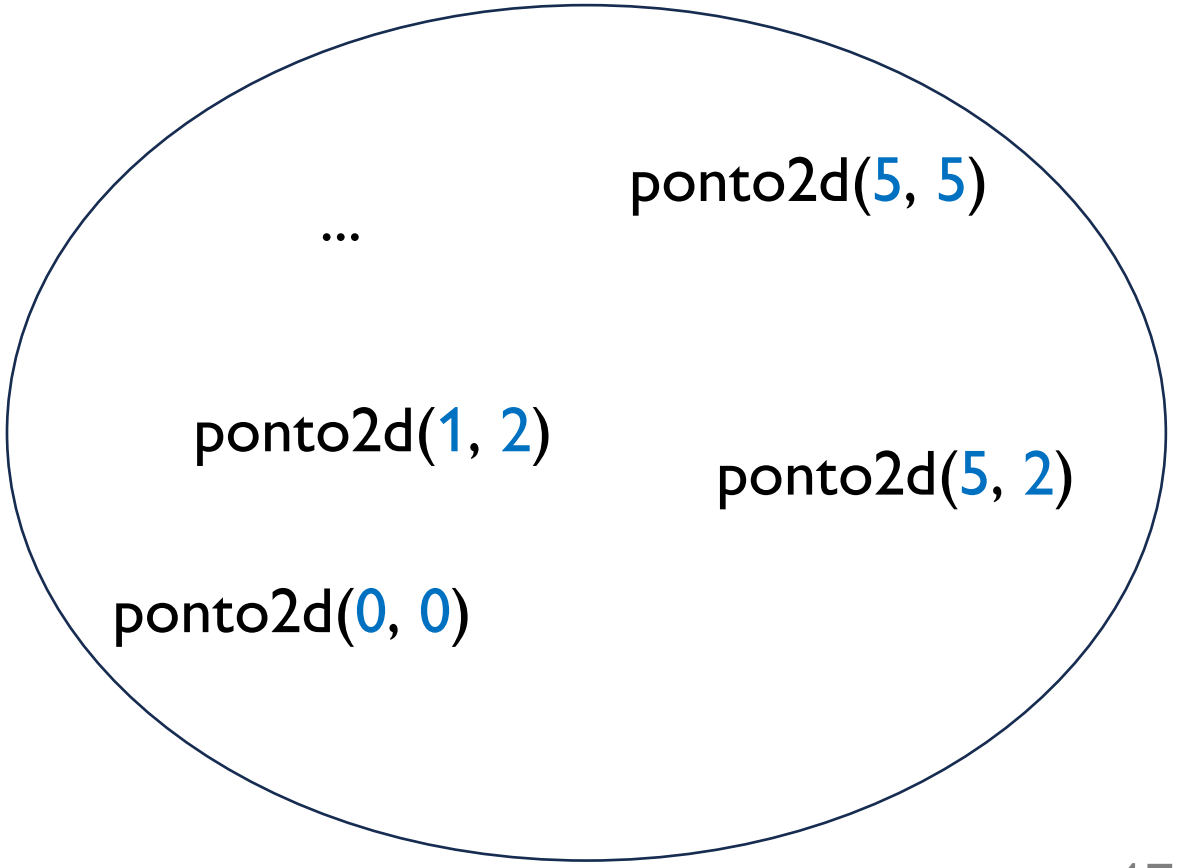
```
data Ponto:
    # Um elemento do conjunto Ponto tem o formato:
    ponto2d(
        coord-x :: Number,  # Coordenada x do ponto
        coord-y :: Number   # Coordenada y do ponto
    )
end
```

5. Interpretação dos atributos

Conjunto Ponto

```
data Ponto:
    ponto2d(coord-x :: Number, coord-y :: Number)
end
```





Construtor

```
data Ponto:
    # Um elemento do conjunto Ponto tem o formato:
    ponto2d(
        coord-x :: Number,  # Coordenada x do ponto
        coord-y :: Number   # Coordenada y do ponto
    )
end
```

- Na definição acima, **ponto2d** é um **construtor** do tipo Ponto.
- A chamada ponto2d(2, 5) cria um novo elemento do tipo Ponto, cujo valor de coord-x é 2 e cujo valor de coord-y é 5.

Seletores de Atributos

Já vimos:

- Como criar um novo tipo estruturado; e
- Como instanciar elementos deste tipo através do construtor.

P. Como acessamos um atributo de um Ponto?

Exemplo:

```
p1 :: Ponto = ponto2d(2, 0)
```

Como acessar a coord-x ou coord-y de p1?

Seletores de Atributos

Exemplo:

```
p1 :: Ponto = ponto2d(2, 0)
```

P. Como acessar a coord-x ou coord-y de p1?

Em Pyret (e em muitas linguagens), podemos usar um seletor de atributo, que é definido por um " . " seguido do nome do atributo:

```
check:
    p1.coord-x is 2
    p1.coord-y is 0
end
```

Seletores de Atributos

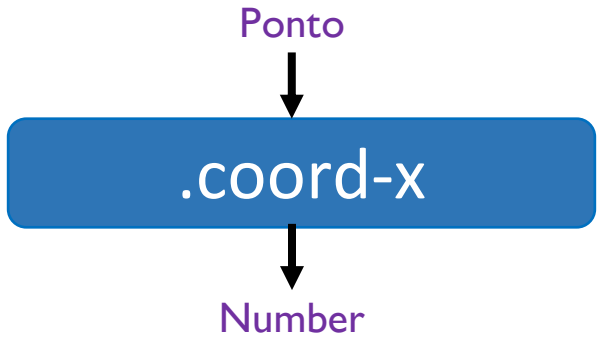
Exemplo:

```
p1 :: Ponto = ponto2d(2, 0)
```

```
check:
    p1.coord-x is 2
    p1.coord-y is 0
end
```

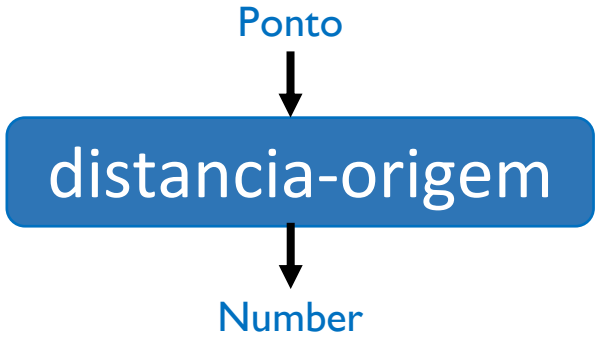
.coord-x é uma sintaxe especial para uma função do tipo:

Ponto → Number



distancia-origem

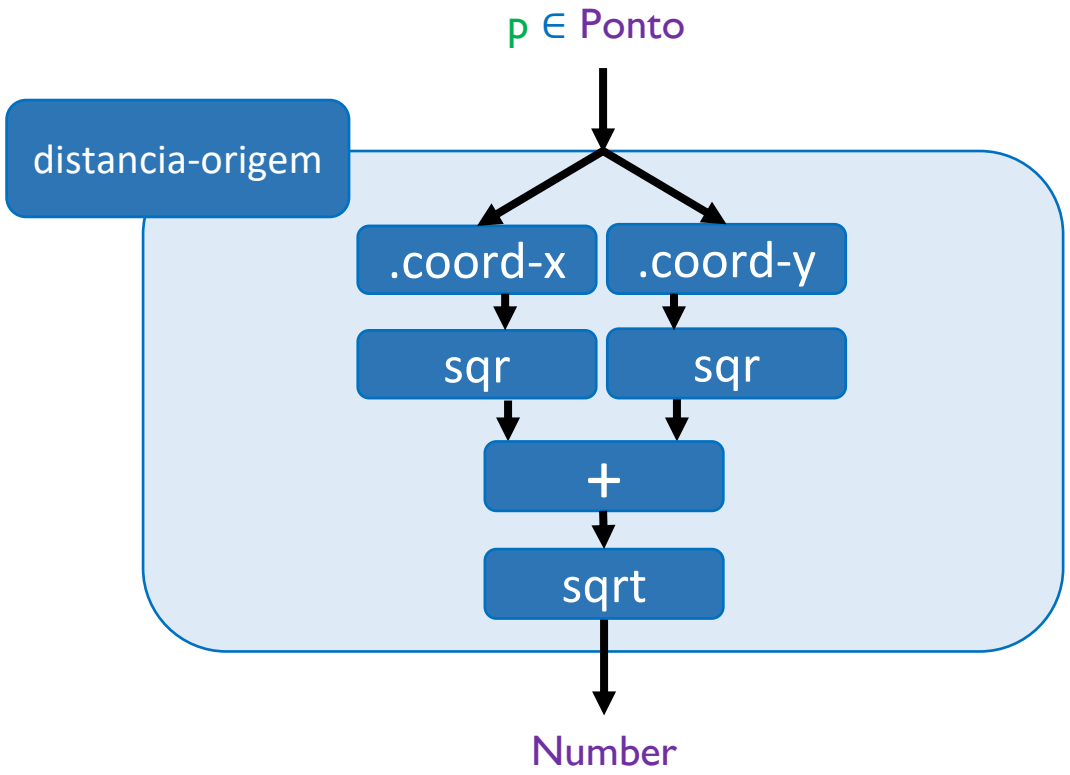
- Dado um ponto no plano, retorna a distância deste ponto à origem (ponto2d(0, 0)).
- Exemplos:
  - distancia-origem(ponto2d(2, 0)) is 2
  - distancia-origem(ponto2d(3, 2)) is 3.6



$$\text{distancia-origem}(\text{ponto}) = \sqrt{(\text{coordenada x do ponto})^2 + (\text{coordenada y do ponto})^2}$$

distancia-origem

$$\text{distancia-origem}(\text{ponto}) = \sqrt{(\text{coordenada x do ponto})^2 + (\text{coordenada y do ponto})^2}$$

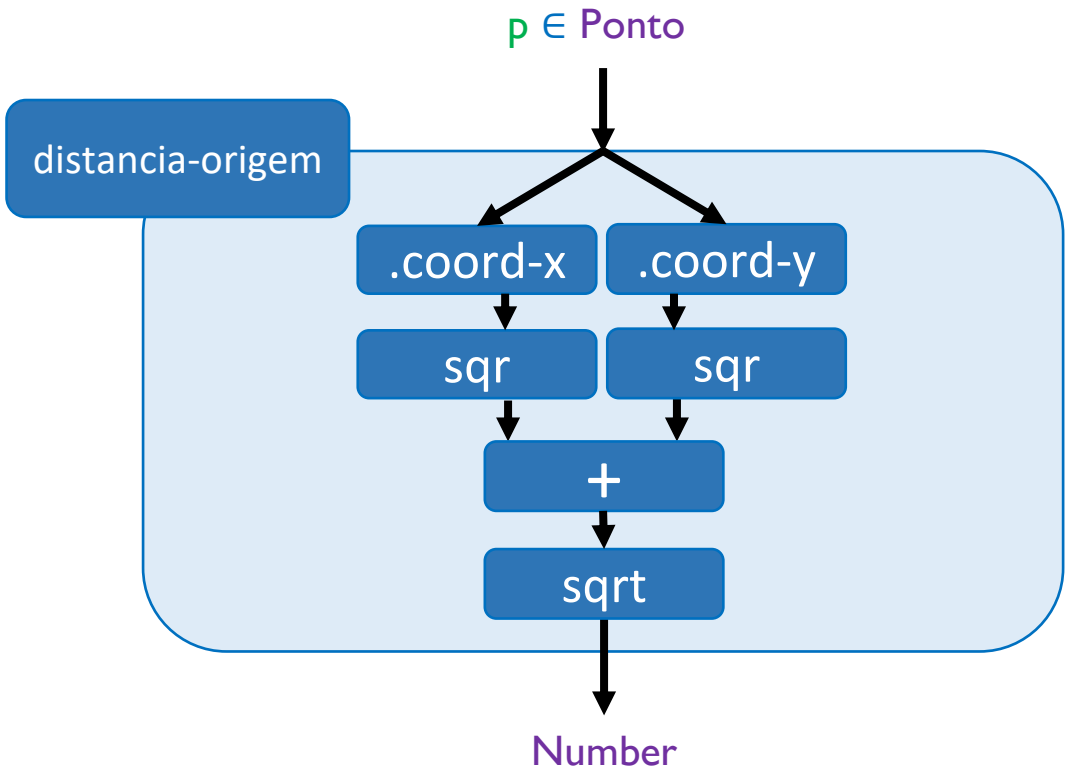



```
fun distancia-origem(p :: Ponto) -> Number:
    doc: "Calcula a distância do ponto p até a
origem (0,0)."


where:
    distancia-origem(ponto2d(2,0)) is 2
    distancia-origem(ponto2d(3,4)) is 5
end
```

distancia-origem

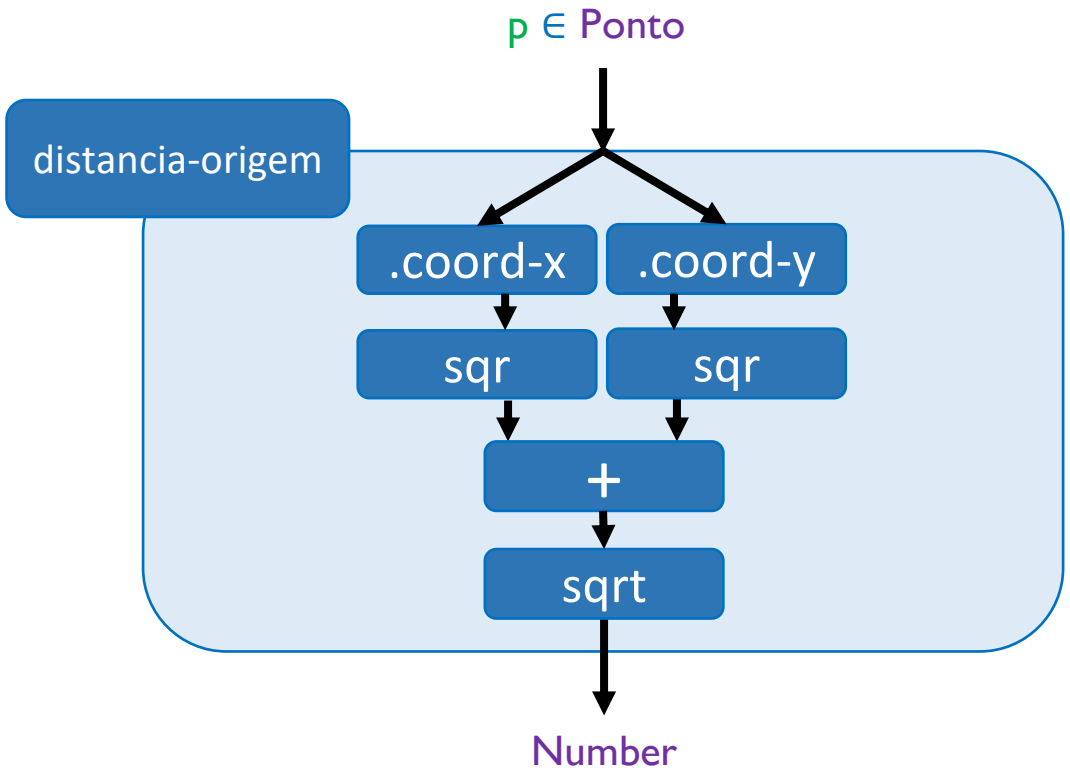
$$\text{distancia-origem}(\text{ponto}) = \sqrt{(\text{coordenada x do ponto})^2 + (\text{coordenada y do ponto})^2}$$




```
fun distancia-origem(p :: Ponto) -> Number:
    doc: "Calcula a distância do ponto p até a
origem (0,0)."
    sqrt( ... )
where:
    distancia-origem(ponto2d(2,0)) is 2
    distancia-origem(ponto2d(3,4)) is 5
end
```

distancia-origem

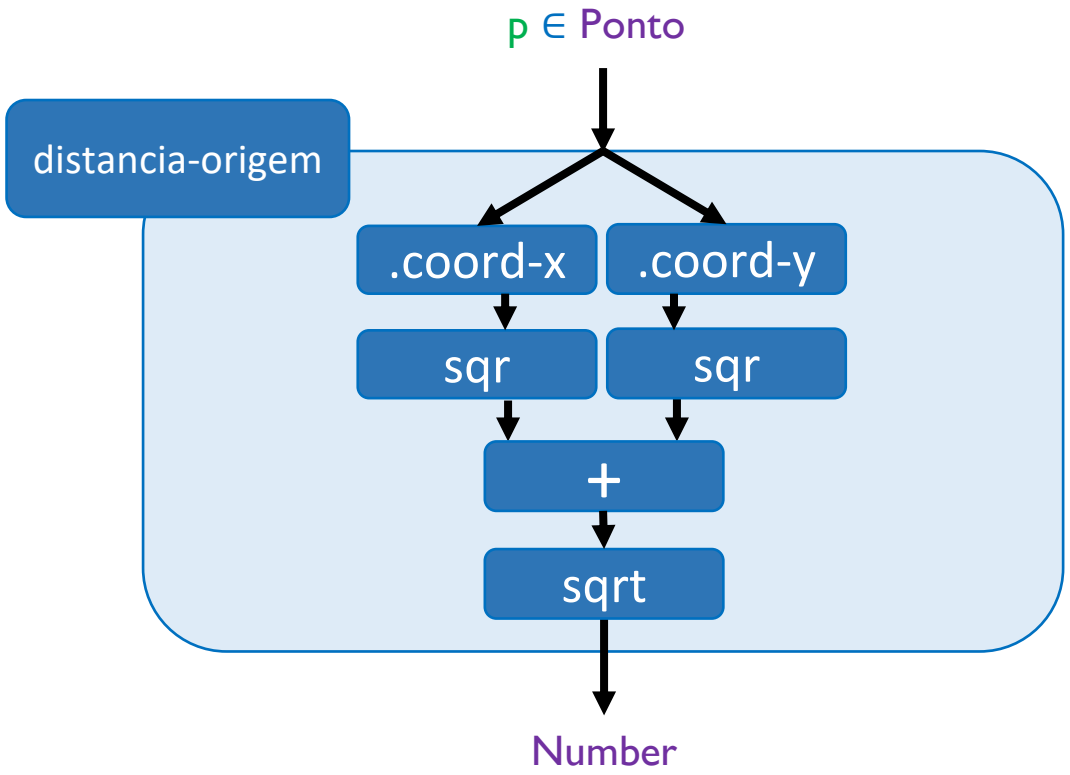
$$\text{distancia-origem}(\text{ponto}) = \sqrt{(\text{coordenada x do ponto})^2 + (\text{coordenada y do ponto})^2}$$




```
fun distancia-origem(p :: Ponto) -> Number:
    doc: "Calcula a distância do ponto p até a
origem (0,0)."
    sqrt(  ... + ... )
where:
    distancia-origem(ponto2d(2,0)) is 2
    distancia-origem(ponto2d(3,4)) is 5
end
```

distancia-origem

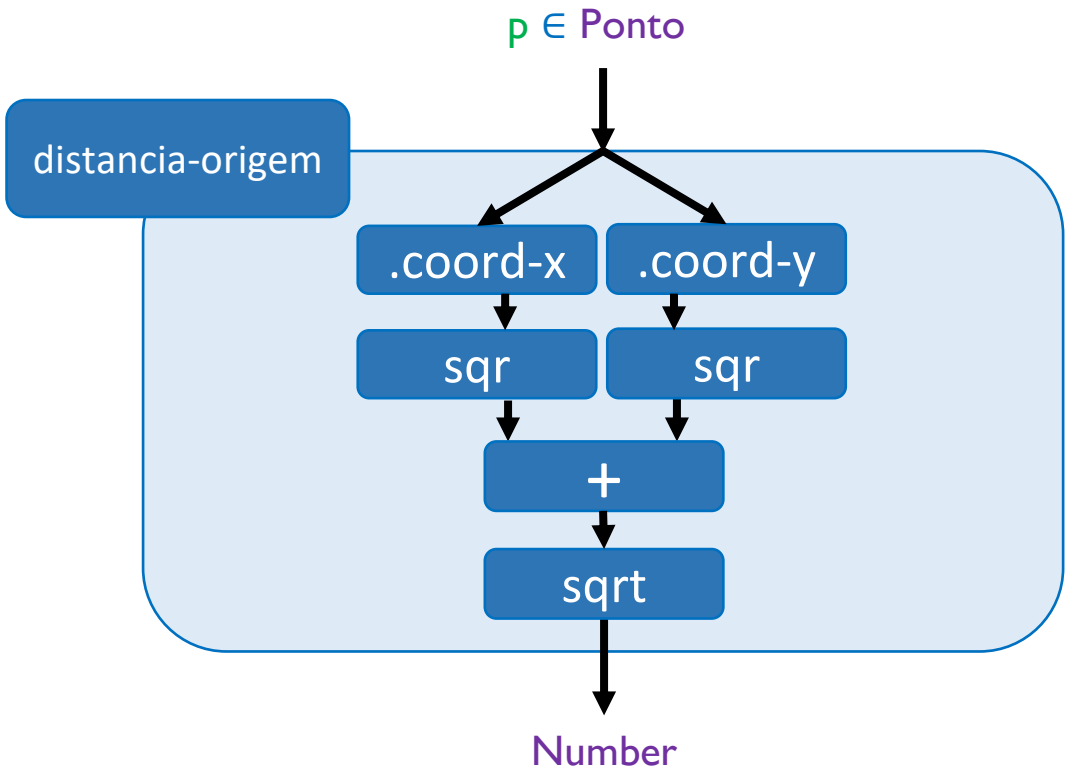
$$\text{distancia-origem}(\text{ponto}) = \sqrt{(\text{coordenada x do ponto})^2 + (\text{coordenada y do ponto})^2}$$




```
fun distancia-origem(p :: Ponto) -> Number:
    doc: "Calcula a distância do ponto p até a
origem (0,0)."
    sqrt(sqr(...) + sqr(...))
where:
    distancia-origem(ponto2d(2,0)) is 2
    distancia-origem(ponto2d(3,4)) is 5
end
```

distancia-origem

$$\text{distancia-origem}(\text{ponto}) = \sqrt{(\text{coordenada x do ponto})^2 + (\text{coordenada y do ponto})^2}$$




```
fun distancia-origem(p :: Ponto) -> Number:
    doc: "Calcula a distância do ponto p até a
origem (0,0)."
    sqrt(sqr(p.coord-x) + sqr(p.coord-y))
where:
    distancia-origem(ponto2d(2,0)) is 2
    distancia-origem(ponto2d(3,4)) is 5
end
```

Definição de Tipos de Dados e Função

Dados

```
data Ponto:
    # Um elemento do conjunto Ponto tem o formato:
    ponto2d(
        coord-x :: Number,  # Coordenada x do ponto
        coord-y :: Number   # Coordenada y do ponto
    )
end
```

Função

```
fun distancia-origem(p :: Ponto) -> Number:
    doc: "Calcula a distância do ponto p até a
origem (0,0)."
    sqrt(sqr(p.coord-x) + sqr(p.coord-y))
where:
    distancia-origem(ponto2d(2,0)) is 2
    distancia-origem(ponto2d(3,4)) is 5
end
```

Exercício

- Escreva uma função distancia-pontos :: (p1 :: Ponto, p2 :: Ponto) → Number, que recebe como entrada dois pontos e calcula a distância entre eles.

- Como podemos reescrever a função distancia-origem usando apenas a função distancia-pontos?

Definição de Estrutura

```
data NomeEstrutura:
    # Um elemento do conjunto NomeEstrutura tem o formato:
    nome-estrutura(
        atr1 :: Tipo1,      # <Interpretação do atr1>
        ...
        atrn :: TipoN       # <Interpretação do atrn>
    )
end
```

Como definir um tipo de dados para Carros?

Modelo	Marca	Ano	Cor	Km
String	String	Number	String	Number
carro				



“GTB”	“Ferrari”	2024	“vermelho”	10.000
carro-ferrari				



“Fusca”	“VW”	1972	“azul”	70.000
carro-fusca				

Conjunto/Tipo Carro

```
data Carro:
    # Um elemento do conjunto Carro tem o formato:
    carro(
        mod :: String, # modelo do carro
        mar :: String, # marca do carro
        ano :: Number, # ano do carro
        cor :: String, # cor do carro
        km :: Number   # quilometragem do carro
    )
end
```

Conjunto/Tipo Carro



“GTB”	“Ferrari”	2024	“vermelho”	10.000
carro-ferrari				

```
ferrari :: Carro = carro("GTB", "Ferrari", 2019, "vermelho", 10000)
```

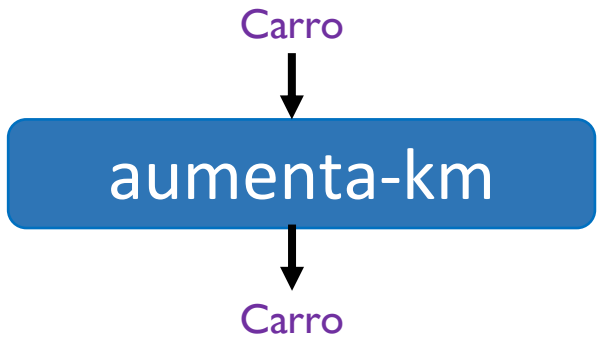


“Fusca”	“VW”	1972	“azul”	70.000
carro-fusca				

```
fusca :: Carro = carro("Fusca", "VW", 1972, "azul", 70000)
```

aumenta-km

Dado um carro, se seu ano for menor que 2013, aumentar quilometragem em 1000.

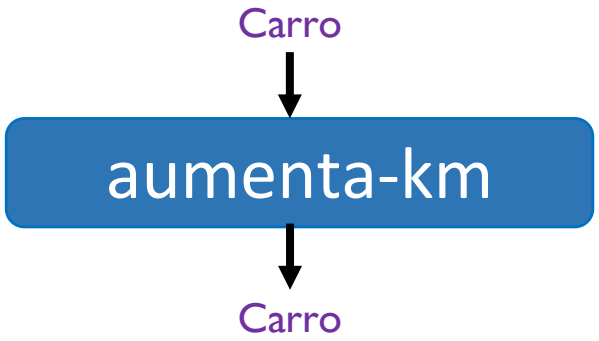



```
aumenta-km( "Fusca" "VW" 1972 "azul" 70.000 )
    is      "Fusca" "VW" 1972 "azul" 71.000

aumenta-km( "GTB" "Ferrari" 2024 "vermelho" 10.000 )
    is      "GTB" "Ferrari" 2024 "vermelho" 10.000
```

aumenta-km

Dado um carro, se seu ano for menor que 2013, aumentar quilometragem em 1000.

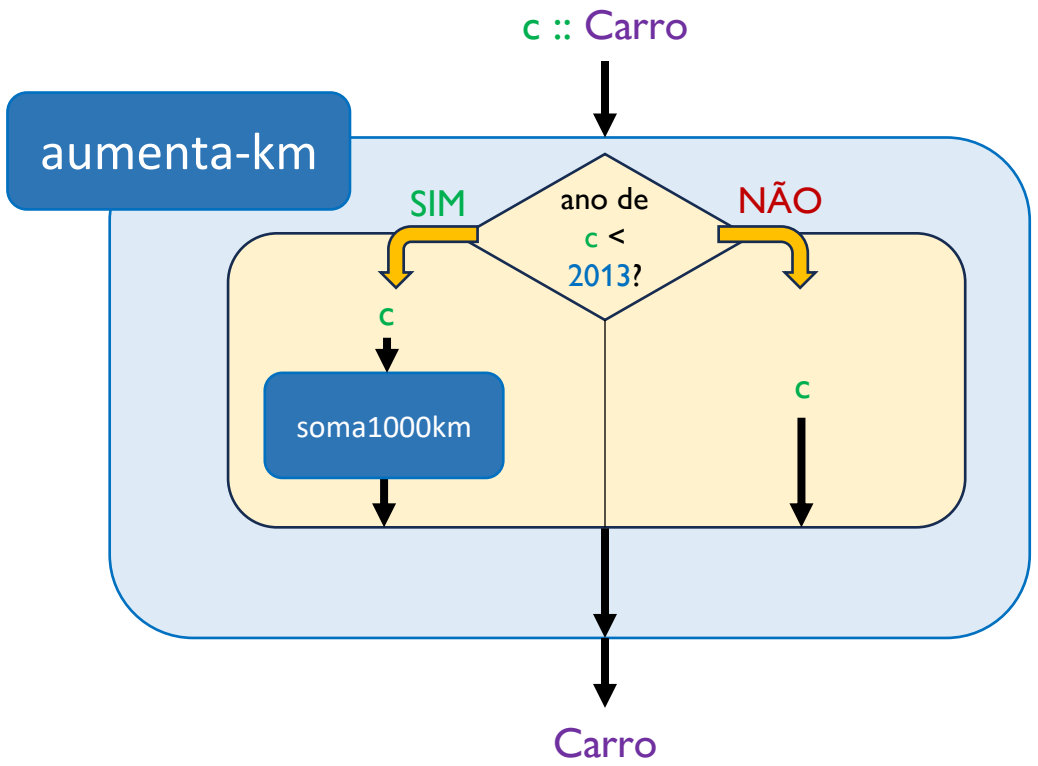



```
where:
    aumenta-km(carro("Fusca", "VW", 1972, "azul", 70000))
       is carro("Fusca", "VW", 1972, "azul", 71000)

    aumenta-km(carro("GTB", "Ferrari", 2019, "vermelho", 10000))
       is carro("GTB", "Ferrari", 2019, "vermelho", 10000)
end
```

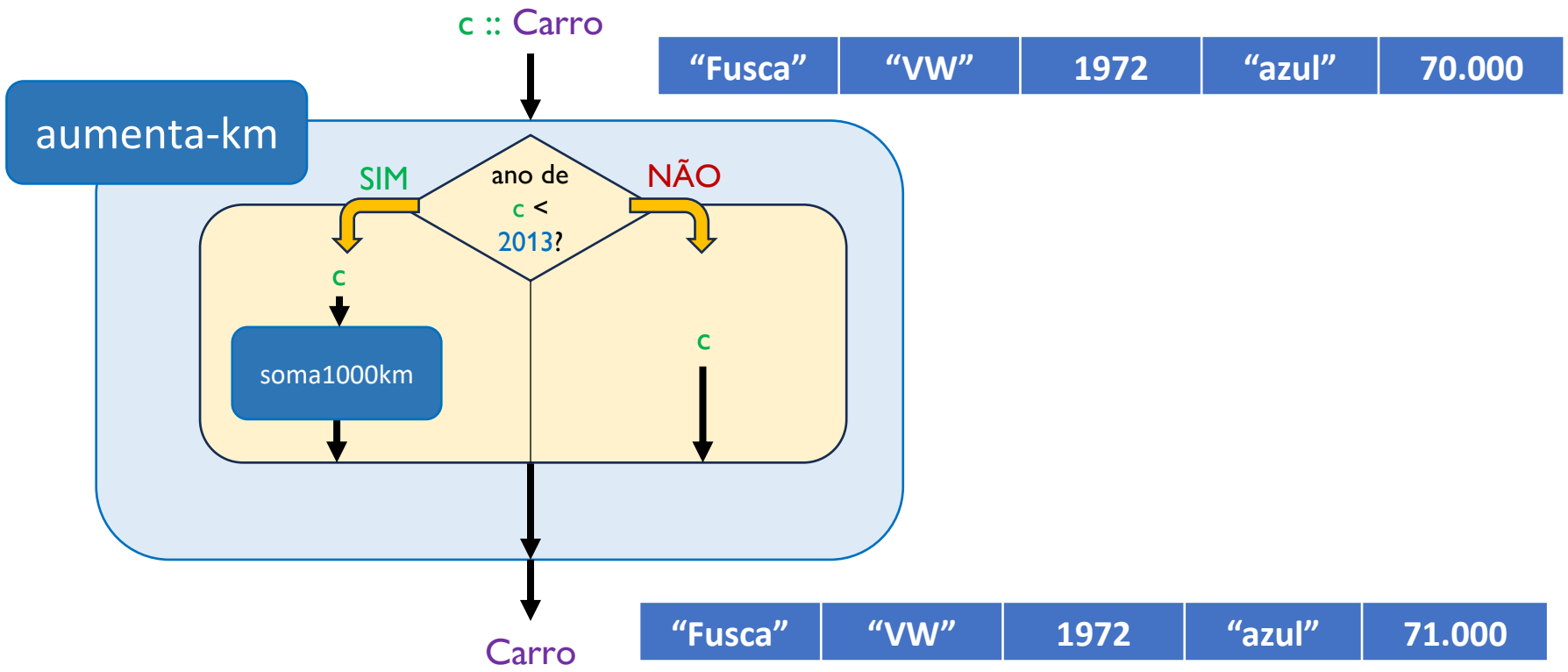
aumenta-km

Dado um carro, se seu ano for menor que 2013, aumentar quilometragem em 1000.



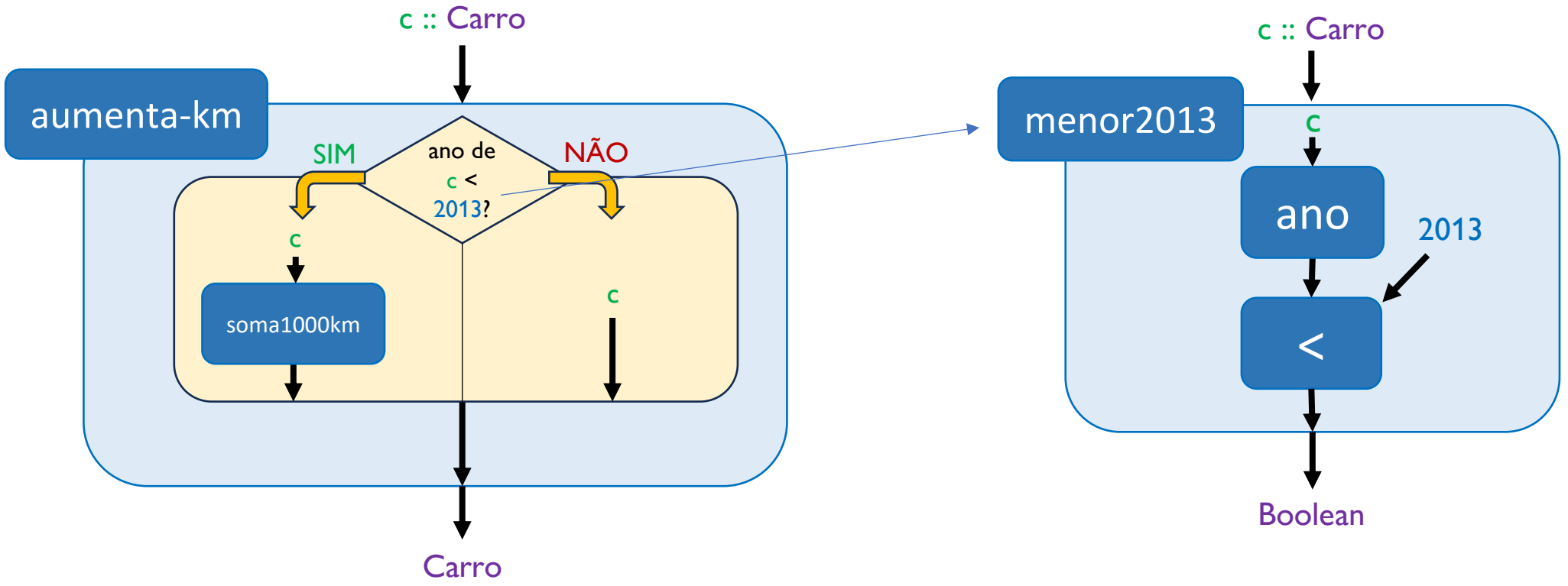
aumenta-km

Dado um carro, se seu ano for menor que 2013, aumentar quilometragem em 1000.



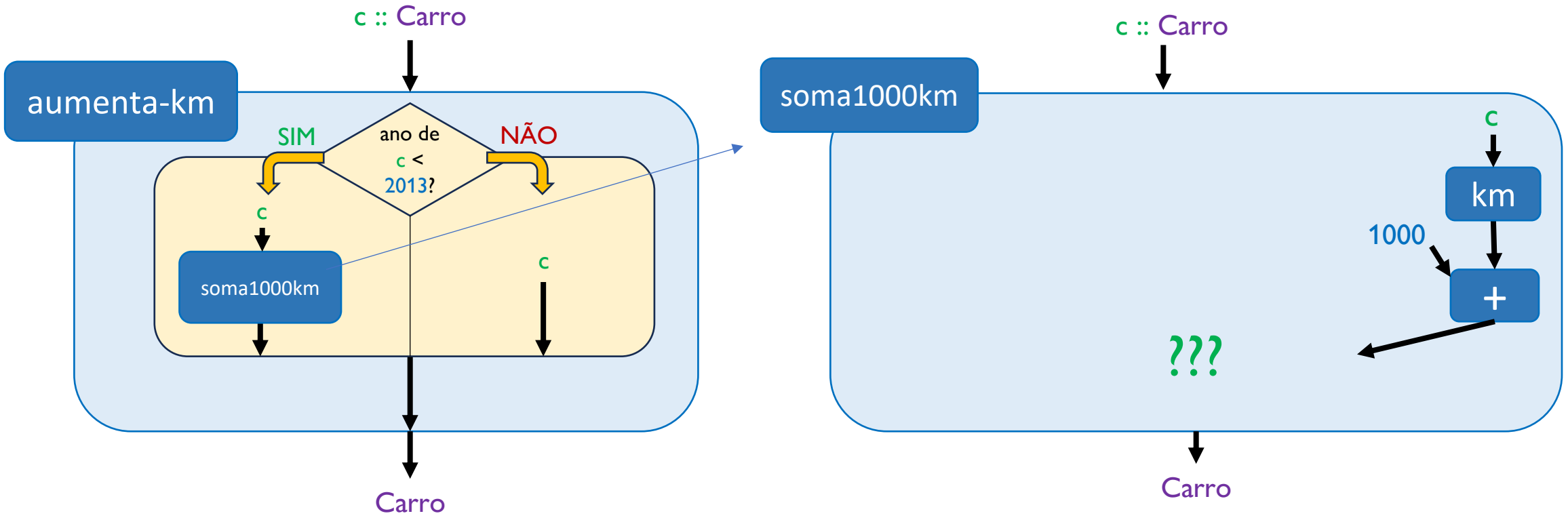
aumenta-km

Dado um carro, se seu ano for menor que 2013, aumentar quilometragem em 1000.



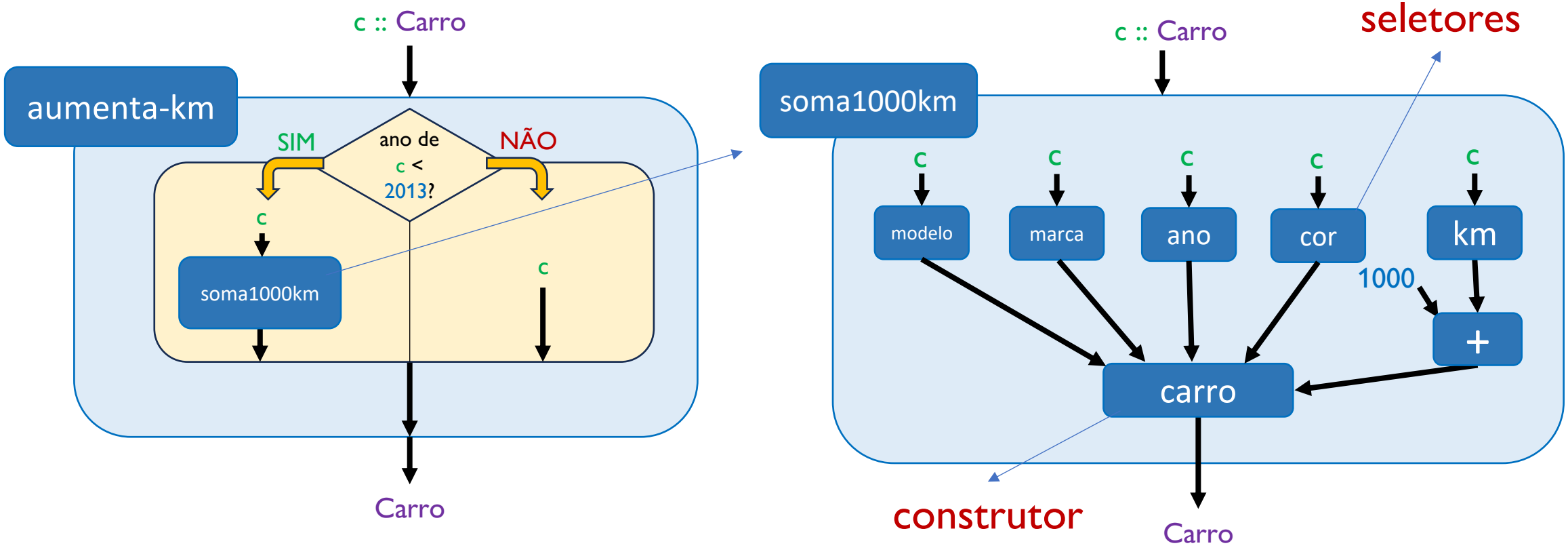
aumenta-km

Dado um carro, se seu ano for menor que 2013, aumentar quilometragem em 1000.



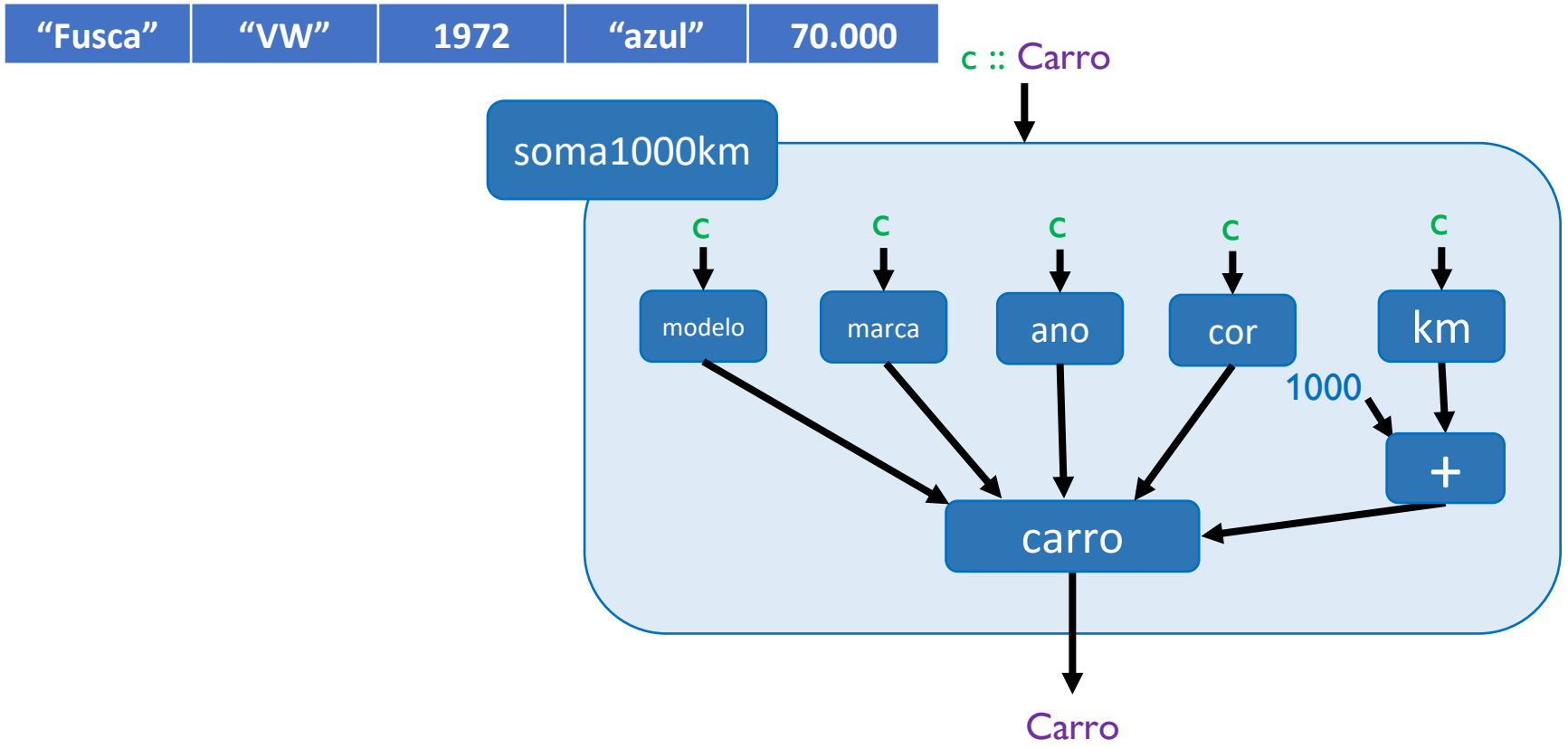
aumenta-km

Dado um carro, se seu ano for menor que 2013, aumentar quilometragem em 1000.



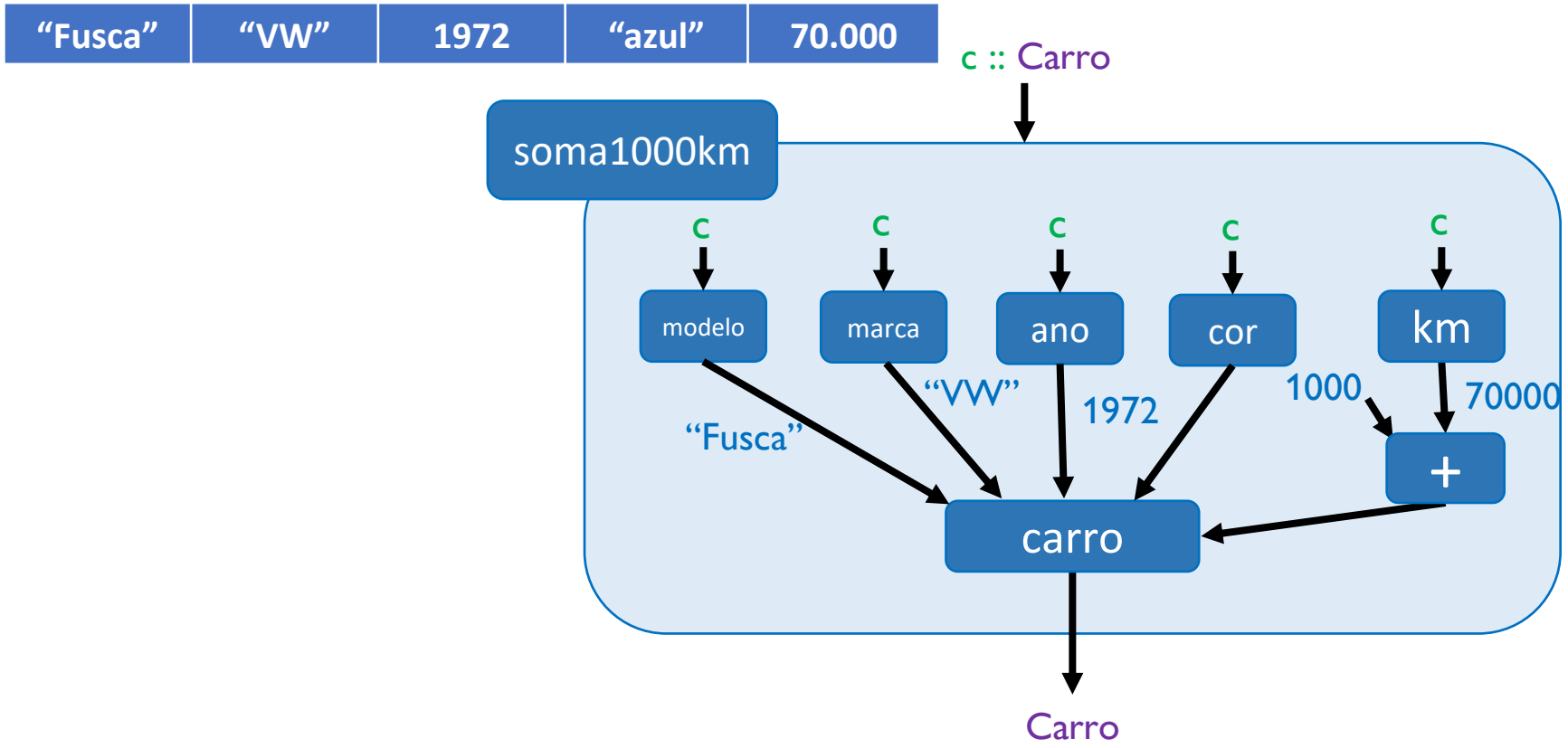
soma1000km

Dado um carro, aumentar quilometragem em 1000.



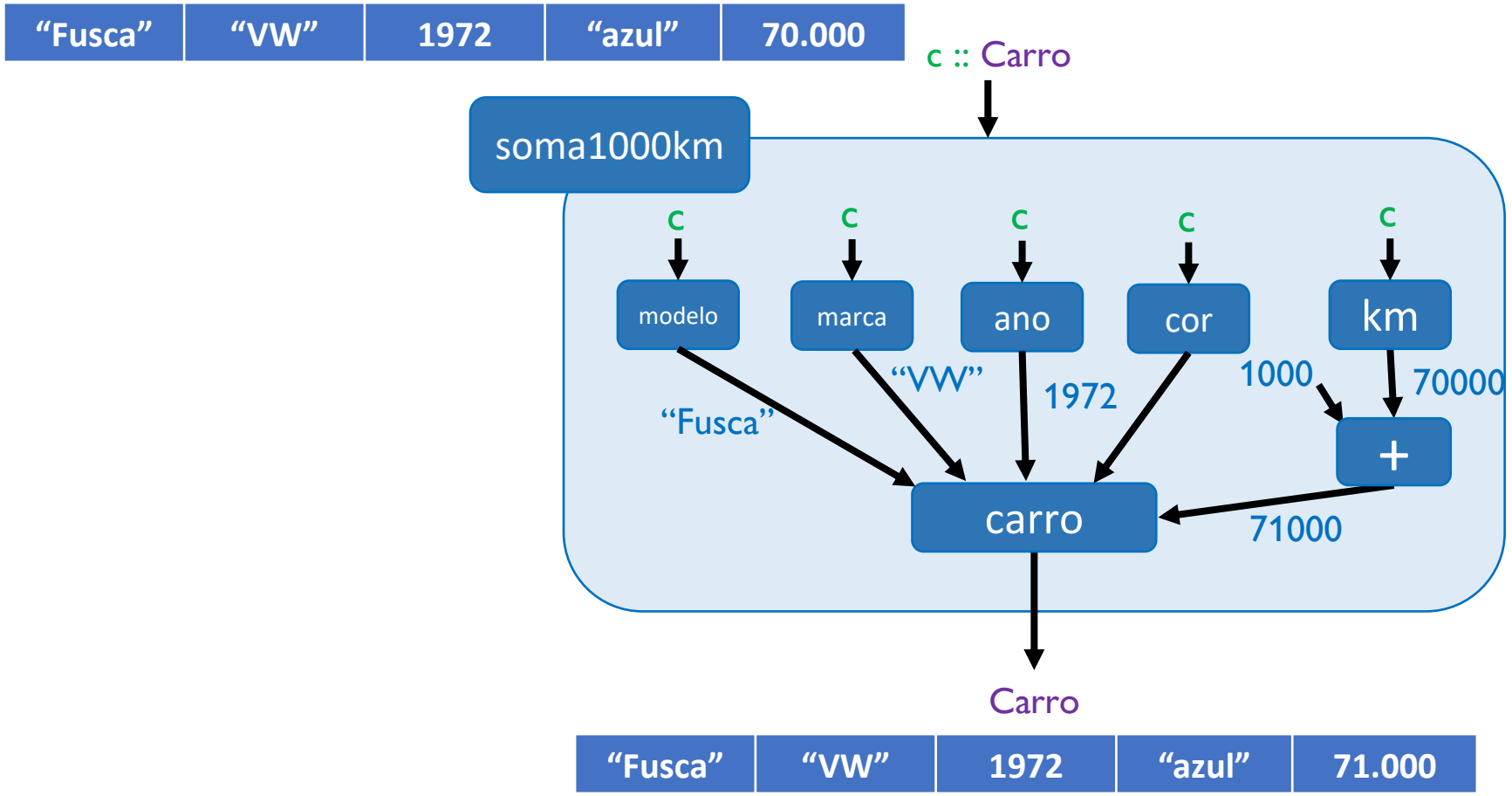
soma1000km

Dado um carro, aumentar quilometragem em 1000.



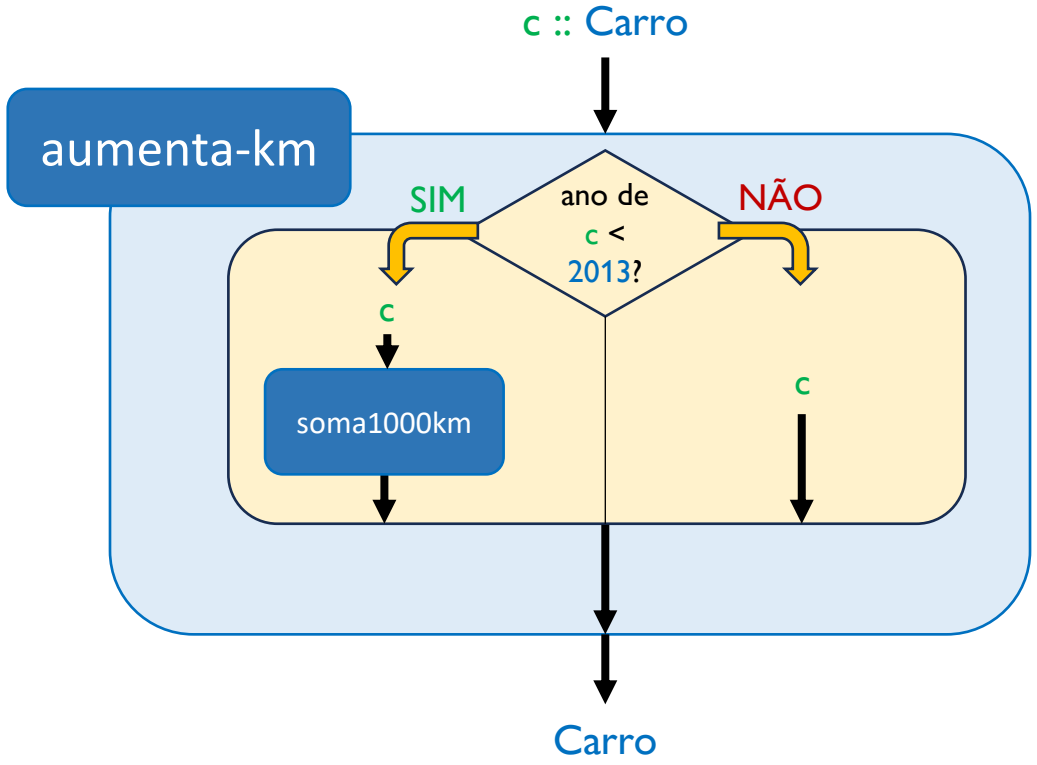
soma1000km

Dado um carro, aumentar quilometragem em 1000.



aumenta-km

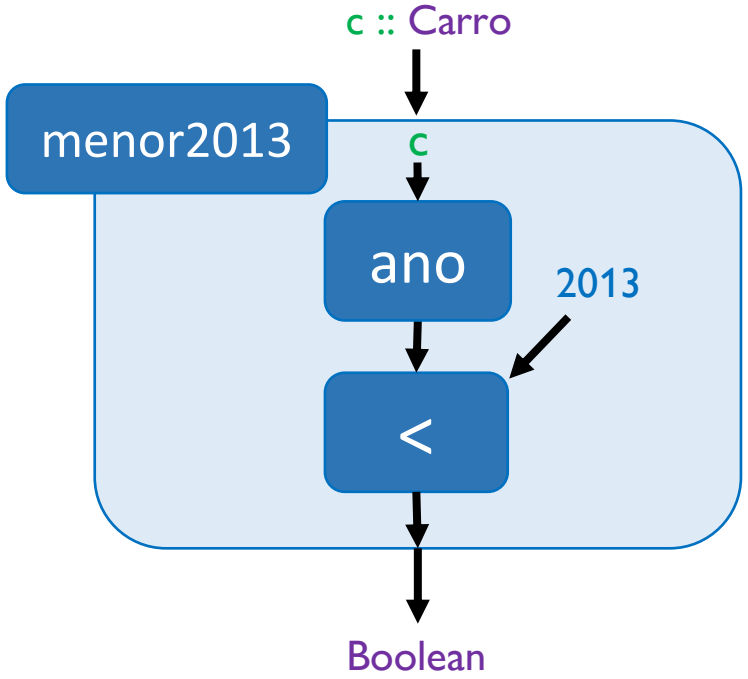
Dado um carro, se seu ano for menor que 2013, aumentar quilometragem em 1000.




```
fun aumenta-km(c :: Carro) -> Carro:
    doc: "Dado um carro, aumenta sua
quilometragem em 1000km se o ano for menor que
2013."
    ask:
    # Se o ano do carro for menor que 2013, soma
    # 1000km em c
    | menor2013(c) then: soma1000km(c)
    # Se não, retorna o carro c sem alterações
    | otherwise: c
    end
end
```

menor2013

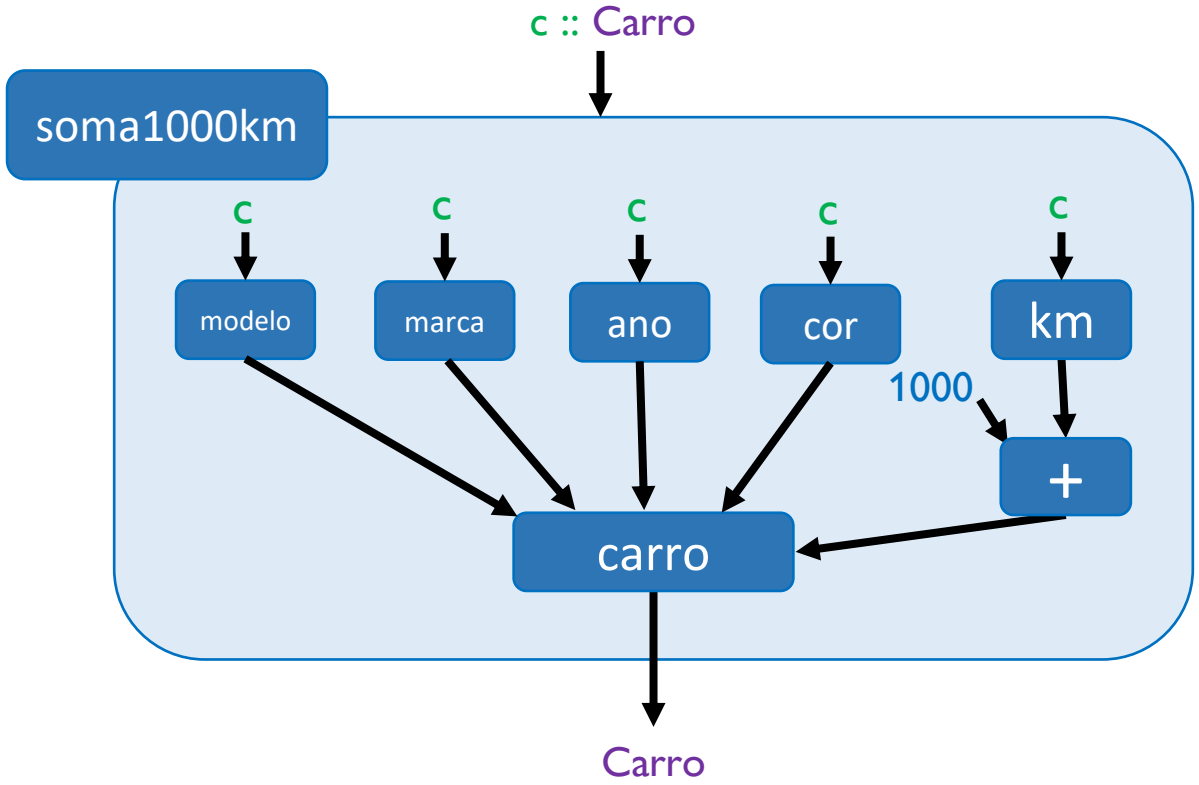
Dado um carro, diz se o seu ano é menor que 2013.




```
fun menor2013(c :: Carro) -> Boolean:
    doc: "Dado um carro, diz se o ano é
menor que 2013."
    c.ano < 2013
end
```

soma1000km

Dado um carro, aumentar quilometragem em 1000.




```
fun soma1000km(c :: Carro) -> Carro:
    doc: "Soma em 1000km a quilometragem
de um carro."
    carro(
        c.mod,
        c.mar,
        c.ano,
        c.cor,
        c.km + 1000
    )
end
```

construtor

seletores

Exercícios

1. Definir a estrutura Musico, que contém o nome, gênero musical e nome da gravadora de um músico.

2. Imagine que a gravadora **Som Livre** encerrou o contrato com alguns artistas e a gravadora **Universal Music** passou a representá-los. Construa uma função que, dado um registro de músico, **altere o nome da gravadora para "Universal Music"** caso a gravadora do músico seja **"Som Livre"**.
   O registro deve permanecer **inalterado** caso a gravadora seja diferente.

3. Modifique a função do exercício 2 para que ela receba como parâmetros, além do registro do músico, o nome da gravadora que está sendo substituída e o nome da nova gravadora.

- Explique: Qual é a vantagem dessa nova versão da função em relação à função do exercício 2?

Dados Estruturados

- Dados de Musico



Nome	Genero	Gravadora
String	String	String
musico		

“Rogério Águas”	“Rock”	“Som Livre”

“Dona Gagá”	“Pop”	“Warner Music”

Exercícios

1. Definir a estrutura Musico, que contém o nome, gênero musical e nome da gravadora de um músico.

```
data Musico:
    # Um elemento do conjunto Musico tem o formato:
    musico(
        nome :: String,          # Nome do músico
        genero :: String,        # Genero musical do artista
        gravadora :: String      # Nome da gravadora
    )
end

# Exemplos de elemento do conjunto Musico
rogerio :: Musico = musico("Rogério Águas", "Rock", "Som Livre")
bruno :: Musico = musico("Bruninho Marte", "Pop", "Warner Music")
gaga :: Musico = musico("Dona Gaga", "Pop", "Interscope Records")
```

Exercícios

Imagine que a gravadora **Som Livre** encerrou o contrato com alguns artistas e a gravadora **Universal Music** passou a representá-los. Construa uma função que, dado um registro de músico, **altere o nome da gravadora para "Universal Music"** caso a gravadora do músico seja **"Som Livre"**. O registro deve permanecer **inalterado** caso a gravadora seja diferente.

```
fun atualiza-gravadora(m :: Musico) -> Musico:
    doc: "Dado um musico, modificada a gravadora para Universal Music caso a gravadora
seja Som Livre."
    ask:
    # Se a gravadora do músico for Som Livre, atualiza para Universal Music
    | (m.gravadora == "Som Livre") then: musico(m.nome, m.genero, "Universal Music")
    # Se não, deixa o músico m sem alterações
    | otherwise: m
    end
where:
    atualiza-gravadora(rogerio) is musico("Rogério Águas", "Rock", "Universal Music")
    atualiza-gravadora(bruno) is musico("Bruninho Marte", "Pop", "Warner Music")
end
```

Exercícios

Imagine que a gravadora **Som Livre** encerrou o contrato com alguns artistas e a gravadora **Universal Music** passou a representá-los. Construa uma função que, dado um registro de músico, **altere o nome da gravadora para "Universal Music"** caso a gravadora do músico seja **"Som Livre"**. O registro deve permanecer **inalterado** caso a gravadora seja diferente.

```
fun atualiza-gravadora(m :: Musico) -> Musico:
    doc: "Dado um musico, modificada a gravadora para Universal Music caso a gravadora
seja Som Livre."
    ask:
    # Se a gravadora do músico for Som Livre, atualiza para Universal Music
    | string-equal(m.gravadora, "Som Livre") then: musico(m.nome, m.genero, "Universal
Music")
    # Se não, deixa o músico m sem alterações
    | otherwise: m
    end
where:
    atualiza-gravadora(rogerio) is musico("Rogério Águas", "Rock", "Universal Music")
    atualiza-gravadora(bruno) is musico("Bruninho Marte", "Pop", "Warner Music")
end
```

Exercícios

```
fun atualiza-gravadora2(m :: Musico, gravadora-antiga :: String, gravadora-nova :: String)
-> Musico:
    doc: "Dado um musico, a gravadora que está fechando e o nome da gravadora nova,
atualiza a gravadora do musico caso ele esteja na gravadora antiga. Caso contrário, não
faz alterações.“

    ask:
    # Se a gravadora do músico for gravadora-antiga, atualiza para gravadora-nova
    | (m.gravadora == gravadora-antiga) then:
       musico(m.nome, m.genero, gravadora-nova)

    # Se não, deixa o músico m sem alterações
    | otherwise: m
    end

end
```

